

Bin-Packing

Enache Stephan-Cristian, Ravariu Maria-Antonia, and Velicu
Alexandru-Ştefan¹

Universitatea Națională de Știință și Tehnologie Politehnica București
<https://upb.ro/>

Abstract. În această lucrare am abordat problema NP-complete Bin-Packing. Aceasta presupune că avem n obiecte, fiecare de o anumită dimensiune, și câteva coșuri de gunoi de capacitate egală. Vrem să repartizăm obiectele în coșurile de gunoi, folosind cât mai puține coșuri posibil. Desigur, dimensiunea totală a obiectelor repartizate într-un coș de gunoi nu trebuie să depășească capacitatea acestuia. Am efectuat și o serie de teste care să confirme corectitudinea și eficacitatea problemei.

Keywords: NP-complete · Bin-Packing · Grafuri.

1 Introducere

1.1 Descrierea problemei

Problema Bin-Packing este o problemă de optimizare în care articole de dimensiuni diferite trebuie ambalate într-un număr finit de containere, fiecare cu o capacitate fixă, astfel încât să se minimizeze numărul de containere utilizate. Această problemă are diverse aplicații, cum ar fi încărcarea containerelor, programarea sarcinilor și plasarea datelor pe mai multe discuri.

Definiția problemei Având n articole cu greutatea diferite și containere cu capacitate c , obiectivul este de a alocă fiecare articol unui container astfel încât numărul total de containere utilizate să fie minimizat. Se presupune că toate articolele au greutatea mai mică decât capacitatea containerului.

Se da un sir de numere pozitive $a_1, \dots, a_n \leq 1$.

Se identifică $k \in \mathbb{N}$ și o funcție de asignare

$$f : \{1, \dots, n\} \rightarrow \{1, \dots, k\}$$

astfel încât, pentru fiecare $j \in \{1, \dots, k\}$, să se verifice

$$\sum_{\substack{i=1 \\ f(i)=j}}^n a_i \leq 1,$$

iar k să fie minim posibil.

1.2 Heuristici pentru Bin Packing

Ipoteza Problema de bin packing este NP-hard, deci este improbabil să existe un algoritm polinomial care să producă întotdeauna soluția optimă. Din acest motiv, folosim *heuristici* care oferă soluții bune, de obicei apropiate de optim.

Algoritmul Next-Fit (NF) Idee:

- Fiecare obiect este încercat în *bin-ul curent*.
- Dacă încape, este plasat acolo.
- Dacă nu încape, se deschide un bin nou.

Next Fit Packing

Raport de aproximație pentru Next-Fit Fie M numărul minim de bin-uri necesare pentru o instanță I .

Theorem 1. *Algoritmul Next Fit folosește cel mult $2M$ bin-uri.*

Proof. Fie $s(B_i)$ suma dimensiunilor obiectelor din bin-ul B_i în soluția NF. Pentru orice doi biți consecutivi B_j și B_{j+1} avem:

$$s(B_j) + s(B_{j+1}) > 1.$$

Dacă NF folosește k bin-uri (presupunem k par), atunci:

$$\sum_{i=1}^k s(B_i) > \frac{k}{2}.$$

Dar suma totală a obiectelor este $\leq M$, deci:

$$\frac{k}{2} < M \quad \Rightarrow \quad k < 2M.$$

Limită inferioară pentru Next-Fit Există secvențe de intrare pentru care Next Fit folosește $2M - 2$ bin-uri, unde M este optimul.

1.3 Algoritmul First-Fit (FF)

- Se scanează bin-urile în ordine.
- Obiectul este plasat în *primul bin* în care încape.
- Se creează un bin nou doar dacă nu încape în niciunul dintre cele existente.

First Fit Packing

Complexitate

- Implementare simplă: $O(n^2)$.
- Implementare rapidă: $O(n \log n)$ folosind un arbore de căutare echilibrat.

Structura nodului arborelui:

- indexul bin-ului,
- capacitatea rămasă,
- valoarea maximă din subarbore.

Raport de aproximație pentru First-Fit

Theorem 2. *Dacă M este numărul optim de bin-uri, First Fit folosește cel mult $\lceil 1.7M \rceil$ bin-uri.*

Există secvențe pentru care First Fit folosește $1.6666 \dots M$ bin-uri.

1.4 Algoritmul Best-Fit (BF)

Idee:

- Obiectul este plasat în bin-ul în care încapă *cel mai strâns* (adică rămâne cel mai puțin spațiu liber).
- Dacă nu încapă în niciun bin, se creează unul nou.

Complexitate: $O(n \log n)$ folosind un arbore echilibrat ordonat după capacitatea rămasă.

2 Heuristici derivate

- **First Fit Decreasing (FFD):** se sortează obiectele descrescător, apoi se aplică FF.
- **Best Fit Decreasing (BFD):** se sortează obiectele descrescător, apoi se aplică BF.

Exemple aplicatii practice Nu există multe probleme de optimizare combinatorie a căror relevanță practică să fie mai evidentă. De exemplu, cea mai simplă versiune a problemei CuttingStock este echivalentă: ni se dau multe grinzi de lungime egală (să zicem 1 metru) și numere $a_1, \dots, a_n$. Vrem să tăiem cât mai puține grinzi în bucăți, astfel încât la final să avem grinzi de lungimi $a_1, \dots, a_n$.

2.1 Aplicarea problemei *Bin Packing* la realizarea unui mix-tape

Problema realizării unui *mix-tape* poate fi modelată în mod natural ca o instanță a problemei clasice de *bin packing*. În acest context:

- fiecare melodie reprezintă un *obiect* (item);
- durata melodiei reprezintă *greutatea* obiectului;
- fiecare casetă (sau fiecare parte a casetei) reprezintă un *bin*;
- capacitatea unui bin este durata maximă care poate fi înregistrată (de exemplu, 30 sau 60 de minute).

Modelare formală Fie o listă de melodii cu durate

$$a_1, a_2, \dots, a_n, \quad a_i > 0.$$

Fiecare melodie trebuie plasată într-un bin (casetă) astfel încât suma duratelor din fiecare bin să nu depășească capacitatea C :

$$\sum_{i: f(i)=j} a_i \leq C, \quad \forall j.$$

Obiectivul este minimizarea numărului total de bin-uri:

$$\min k.$$

Aceasta este exact formularea problemei de *bin packing*.

Interpretare intuitivă Realizarea unui mix-tape presupune alegerea unui set de melodii care să încapă într-o durată fixă. Dacă numărul melodiilor este mare sau dacă se dorește realizarea mai multor mixuri, apare necesitatea împărțirii lor în mai multe casete. Astfel, problema devine:

Cum putem împacheta melodiile în cât mai puține casete, fără a depăși durata maximă a fiecăreia?

Heuristici utilizabile Heuristicile clasice de bin packing se aplică direct:

- **Next Fit**: se adaugă melodiile în caseta curentă până nu mai încap nimic, apoi se trece la următoarea.
- **First Fit**: fiecare melodie este plasată în prima casetă în care încap.
- **Best Fit**: melodia este plasată în caseta în care rămâne cel mai puțin spațiu liber după adăugare.
- **First Fit Decreasing** / **Best Fit Decreasing**: melodiile sunt sortate descrescător după durată înainte de împachetare, ceea ce produce rezultate mult mai bune în practică.

3 Reducerea Partition la Bin Packing (decizie)

3.1 Problema Partition

Fie o mulțime A de numere pozitive. Putem împărți A în două submulțimi disjuncte A_1 și A_2 astfel încât:

$$\sum_{a_i \in A_1} a_i = \sum_{a_j \in A_2} a_j?$$

4 Demonstrație NP-Hard

4.1 Problema Bin Packing (decizie)

Fie o mulțime B de N elemente pozitive și o capacitate C . Putem distribui elementele în K binuri astfel încât:

$$\sum_{b_i \in B_j} b_i \leq C, \quad \text{pentru fiecare bin } B_j?$$

4.2 Reducere

Construim o instanță de Bin Packing pornind de la Partition:

- $B = A$ (elementele rămân neschimbate)
- $C = \frac{\sum a_i}{2}$
- $K = 2$

” $\Rightarrow$ ”:

Dacă Partition are soluție, atunci există două submulțimi A_1 și A_2 cu sume egale:

$$\sum_{a_i \in A_1} a_i = \sum_{a_j \in A_2} a_j = \frac{\sum a_k}{2}$$

Putem, deci, distribui elementele în exact două binuri de capacitate $C \Rightarrow$ Bin Packing = TRUE.

” $\Leftarrow$ ”:

Analog, dacă Bin Packing are soluție cu $K = 2$ și $C = \frac{\sum a_i}{2}$, atunci cele două binuri sunt umplute exact $\Rightarrow$ sumele sunt egale $\Rightarrow$ Partition = TRUE.

4.3 Concluzie

$$\text{Partition} \leq_p \text{Bin Packing (decizie)}$$

5 Reducerea 3-Partition la Bin Packing (decizie)

5.1 Problema 3-Partition

Fie o mulțime A de numere pozitive, cu $|A| = 3N$. Putem împărți A în N triplete disjuncte astfel încât fiecare triplet să aibă aceeași sumă?

$$S_{\text{triplet}} = \frac{\sum a_i}{N}$$

5.2 Construirea instanței de Bin Packing

- $B = A$
- $C = \frac{\sum a_i}{N} = S_{\text{triplet}}$
- $K = N = \frac{|B|}{3}$

5.3 Implicație

” $\Leftarrow$ ”:

Dacă 3-Partition are soluție $\Rightarrow$ putem grupa elementele în N triplete de sumă $C \Rightarrow$ fiecare triplet intră într-un bin, ocupându-l în totalitate $\Rightarrow$ Bin Packing are soluție cu $K = N$.

” $\Rightarrow$ ”:

Analog, dacă Bin Packing are soluție cu $K = N$ și capacitate C , iar suma totală este $N \cdot C$, atunci fiecare bin este umplut cu exact C , iar fiecare bin conține exact 3 elemente $\Rightarrow$ avem o soluție pentru 3-Partition.

5.4 Concluzie

$$3\text{-Partition} \leq_p \text{Bin Packing (decizie)}$$

Bin Packing (decizie) este NP-completă.

6 Prezentare Algoritmi

Programul implementează două categorii principale de algoritmi pentru problema *Bin Packing*: algoritmi exacti bazați pe backtracking și algoritmi euristici (First Fit, Best Fit și variantele lor Decreasing).

6.1 Descrierea modului de funcționare

Algoritmi de Backtracking Algoritmii de backtracking explorează toate modulele posibile de a plasa pachetele în binuri. Fiecare stare a căutării este reprezentată prin structura:

```
struct SearchState {
    int item_idx;
    int bin_idx;
};
```

Această structură reține poziția curentă în arborele de căutare:

- `item_idx` indică pachetul curent care trebuie plasat,
- `bin_idx` indică binul în care se încearcă plasarea.

Plasarea unui pachet în binurile existente

```
if (curr.bin_idx < (int)current_bins.size()) {
    if (current_bins[curr.bin_idx].space_left >= weight) {
        current_bins[curr.bin_idx].space_left -= weight;
        current_bins[curr.bin_idx].packets.push_back({weight, curr.item_idx});
        search_stack.push_back({curr.item_idx + 1, 0});
    } else {
        curr.bin_idx++;
    }
}
```

Explicație: Dacă `bin_idx` este mai mic decât numărul de binuri existente, algoritmul încearcă să plaseze pachetul în binul respectiv. Dacă încape, se actualizează spațiul liber și se trece la următorul pachet. Dacă nu încape, se incrementează `bin_idx` pentru a încerca următorul bin.

Deschiderea unui bin nou

```
else if (curr.bin_idx == (int)current_bins.size()) {
    current_bins.push_back({capacity - weight, {{weight, curr.item_idx}}});
    search_stack.push_back({curr.item_idx + 1, 0});
}
```

Explicație: Dacă toate binurile existente au fost încercate, algoritmul deschide un bin nou și plasează pachetul acolo. Apoi continuă cu următorul pachet.

Backtracking: revenirea la starea anterioară

```
void undo_move(vector<bin>& bins, int bin_idx, int weight) {
    bins[bin_idx].space_left += weight;
    bins[bin_idx].packets.pop_back();
    if (bins[bin_idx].packets.empty()) {
        bins.pop_back();
    }
}
```

Explicație: Această funcție inversează ultima mutare:

- readaugă greutatea în spațiul liber,
- elimină pachetul din bin,
- dacă binul a rămas gol, este eliminat complet.

Backtracking asupra ultimului pachet

```
void backtrack_last_item(vector<bin>& bins, vector<SearchState>& stack, vector<packet> pa
SearchState& prev = stack.back();
int weight = pack[prev.item_idx].weight;
if (prev.bin_idx < (int)bins.size()) {
    undo_move(bins, prev.bin_idx, weight);
} else {
    bins.pop_back();
}
prev.bin_idx++;
}
```

Explicație: Când o ramură a căutării este epuizată, algoritmul:

1. scoate pachetul plasat anterior,
2. incrementează bin_idx pentru a încerca următoarea opțiune,
3. continuă explorarea.

Algoritmi Euristici

First Fit (FF)

```
for(size_t j = 0; j < bins.size(); j++) {
    if(bins[j].space_left >= pack[i].weight) {
        bins[j].space_left -= pack[i].weight;
        bins[j].packets.push_back(pack[i]);
        fit = true;
        break;
    }
}
```

Explicație: Algoritmul parcurge binurile în ordine și plasează pachetul în primul bin în care încap. Este rapid, dar poate produce soluții suboptimale.

Best Fit (BF)

```
if(bins[j].space_left > pack[i].weight &&
bins[j].space_left - pack[i].weight < remaining_space_after_fit) {
    best_index = j;
    remaining_space_after_fit = bins[j].space_left - pack[i].weight;
}
```


Explicație: Algoritmul caută binul în care pachetul va lăsa cel mai puțin spațiu liber. Aceasta minimizează fragmentarea și produce soluții mai bune decât FF.

Variantele Decreasing

```
sort(pack.begin(), pack.end(), compare_packs_decr);
return first_fit(cap, pack);
```

Explicație: Sortarea descrescătoare reduce dramatic riscul de fragmentare. FFD și BFD sunt printre cele mai eficiente euristici clasice.

6.2 Analiza complexității soluțiilor

Backtracking Complexitatea în timp este exponențială:

$$O(k^n)$$

Complexitatea în spațiu:

$$O(n)$$

First Fit / Best Fit Complexitatea în timp:

$$O(n \cdot m)$$

Variantele Decreasing:

$$O(n \log n + n^2)$$

Complexitatea în spațiu:

$$O(n)$$

6.3 Avantaje și dezavantaje

Backtracking Avantaje:

- garantează soluția optimă;
- poate genera toate soluțiile optime;
- pruning-ul reduce semnificativ timpul de execuție.

Dezavantaje:

- complexitate exponențială;
- timp mare de execuție pentru seturi mari;
- implementare mai complexă.

First Fit / Best Fit Avantaje:

- foarte rapide și ușor de implementat;
- potrivite pentru seturi mari de date;
- variantele Decreasing oferă rezultate bune în practică.

Dezavantaje:

- nu garantează soluția optimă;
- pot produce soluții suboptimale;
- sensibile la ordinea pachetelor.